Red Hat Developer Hub 1.6

Configuring Red Hat Developer Hub

Adding custom config maps and secrets to configure your Red Hat Developer Hub instance to work in your IT ecosystem

Last Updated: 2025-08-12

Red Hat Developer Hub 1.6 Configuring Red Hat Developer Hub

Adding custom config maps and secrets to configure your Red Hat Developer Hub instance to work in your IT ecosystem

Legal Notice

Copyright © 2025 Red Hat, Inc.

The text of and illustrations in this document are licensed by Red Hat under a Creative Commons Attribution–Share Alike 3.0 Unported license ("CC-BY-SA"). An explanation of CC-BY-SA is available at
http://creativecommons.org/licenses/by-sa/3.0/
. In accordance with CC-BY-SA, if you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, Red Hat Enterprise Linux, the Shadowman logo, the Red Hat logo, JBoss, OpenShift, Fedora, the Infinity logo, and RHCE are trademarks of Red Hat, Inc., registered in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

Java ® is a registered trademark of Oracle and/or its affiliates.

XFS ® is a trademark of Silicon Graphics International Corp. or its subsidiaries in the United States and/or other countries.

MySQL ® is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js ® is an official trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack ® Word Mark and OpenStack logo are either registered trademarks/service marks or trademarks/service marks of the OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. We are not affiliated with, endorsed or sponsored by the OpenStack Foundation, or the OpenStack community.

All other trademarks are the property of their respective owners.

Abstract

Learn how to configure Red Hat Developer Hub (RHDH) for production to work in your IT ecosystem by adding custom config maps and secrets.

Table of Contents

PREFACE 3

CHAPTER 1. PROVISIONING AND USING YOUR CUSTOM RED HAT DEVELOPER HUB CONFIGURATION 4
1.1. PROVISIONING YOUR CUSTOM RED HAT DEVELOPER HUB CONFIGURATION 4
1.2. USING THE RED HAT DEVELOPER HUB OPERATOR TO RUN DEVELOPER HUB WITH YOUR CUSTOM CONFIGURATION 6
1.2.1. Mounting additional files in your custom configuration using the Red Hat Developer Hub Operator 10
1.3. USING THE RED HAT DEVELOPER HUB HELM CHART TO RUN DEVELOPER HUB WITH YOUR CUSTOM CONFIGURATION 11

CHAPTER 2. CONFIGURING EXTERNAL POSTGRESQL DATABASES 13
2.1. CONFIGURING AN EXTERNAL POSTGRESQL INSTANCE USING THE OPERATOR 13
2.2. CONFIGURING AN EXTERNAL POSTGRESQL INSTANCE USING THE HELM CHART 15
2.3. MIGRATING LOCAL DATABASES TO AN EXTERNAL DATABASE SERVER USING THE OPERATOR 19

CHAPTER 3. CONFIGURING RED HAT DEVELOPER HUB DEPLOYMENT WHEN USING THE OPERATOR 23

CHAPTER 4. CONFIGURING READONLYROOTFILESYSTEM IN RED HAT DEVELOPER HUB 26
4.1. CONFIGURING THE READONLYROOTFILESYSTEM OPTION IN A RED HAT DEVELOPER HUB OPERATOR DEPLOYMENT 26
4.2. CONFIGURING THE READONLYROOTFILESYSTEM OPTION IN A RED HAT DEVELOPER HUB HELM CHART DEPLOYMENT 26

CHAPTER 5. CONFIGURING HIGH AVAILABILITY IN RED HAT DEVELOPER HUB 27
5.1. CONFIGURING HIGH AVAILABILITY IN A RED HAT DEVELOPER HUB OPERATOR DEPLOYMENT 27
5.2. CONFIGURING HIGH AVAILABILITY IN A RED HAT DEVELOPER HUB HELM CHART DEPLOYMENT 28

CHAPTER 6. RUNNING RED HAT DEVELOPER HUB BEHIND A CORPORATE PROXY 29
6.1. UNDERSTANDING THE NO_PROXY EXCLUSION RULES 29
6.2. CONFIGURING PROXY INFORMATION IN OPERATOR DEPLOYMENT 30
6.3. CONFIGURING PROXY INFORMATION IN HELM DEPLOYMENT 31

CHAPTER 7. CONFIGURING AN RHDH INSTANCE WITH A TLS CONNECTION IN KUBERNETES 33

CHAPTER 8. USING THE DYNAMIC PLUGINS CACHE 36
8.1. USING THE DYNAMIC PLUGINS CACHE 36
8.2. CREATING A PVC FOR THE DYNAMIC PLUGIN CACHE BY USING THE OPERATOR 36
8.3. CREATING A PVC FOR THE DYNAMIC PLUGIN CACHE USING THE HELM CHART 37
8.4. CONFIGURING THE DYNAMIC PLUGINS CACHE 38

CHAPTER 9. CONFIGURING DEFAULT MOUNTS FOR SECRETS AND PVCS 40
9.1. CONFIGURING MOUNT PATHS FOR SECRETS AND PVCS 40
9.2. MOUNTING SECRETS AND PVCS TO SPECIFIC CONTAINERS 41

CHAPTER 10. ENABLING THE RED HAT DEVELOPER HUB PLUGIN ASSETS CACHE 42

PREFACE

Learn how to configure Red Hat Developer Hub (RHDH) for production to work in your IT ecosystem by adding custom config maps and secrets.

CHAPTER 1. PROVISIONING AND USING YOUR CUSTOM RED HAT DEVELOPER HUB CONFIGURATION

To configure Red Hat Developer Hub, use these methods, which are widely used to configure a Red Hat OpenShift Container Platform application:

- Use config maps to mount files and directories.
- Use secrets to inject environment variables.

Learn to apply these methods to Developer Hub:

1. Provision your custom config maps and secrets to OpenShift Container Platform .
2. Use your selected deployment method to mount the config maps and inject the secrets:
   - Use the Red Hat Developer Hub operator to deploy Developer Hub .
   - Use the Red Hat Developer Hub Helm chart to deploy Developer Hub .

1.1. PROVISIONING YOUR CUSTOM RED HAT DEVELOPER HUB CONFIGURATION

To configure Red Hat Developer Hub, provision your custom Red Hat Developer Hub config maps and secrets to Red Hat OpenShift Container Platform before running Red Hat Developer Hub.

TIP

You can skip this step to run Developer Hub with the default config map and secret. Your changes on this configuration might get reverted on Developer Hub restart.

Prerequisites

- By using the OpenShift CLI (**oc**), you have access, with developer permissions, to the OpenShift Container Platform cluster aimed at containing your Developer Hub instance.

Procedure

1. Author your custom ***<my_product_secrets>*.txt** file to provision your secrets as environment variables values in an OpenShift Container Platform secret, rather than in clear text in your configuration files. It contains one secret per line in **KEY=value** form.
   - Enter your authentication secrets.
2. Author your custom **app-config.yaml** file. This is the main Developer Hub configuration file. You need a custom **app-config.yaml** file to avoid the Developer Hub installer to revert user edits during upgrades. When your custom **app-config.yaml** file is empty, Developer Hub is using default values.
   - To prepare a deployment with the Red Hat Developer Hub Operator on OpenShift Container Platform, you can start with an empty file.
   - To prepare a deployment with the Red Hat Developer Hub Helm chart, or on Kubernetes, enter the Developer Hub base URL in the relevant fields in your **app-config.yaml** file to

ensure proper functionality of Developer Hub. The base URL is what a Developer Hub user sees in their browser when accessing Developer Hub. The relevant fields are **baseUrl** in the **app** and **backend** sections, and **origin** in the **backend.cors** subsection:

Example 1.1. Configuring the baseUrl in app-config.yaml

```
app:
  title: Red Hat Developer Hub
  baseUrl: https://<my_developer_hub_url>

backend:
  auth:
    externalAccess:
      - type: legacy
        options:
          subject: legacy-default-config
          secret: "${BACKEND_SECRET}"
  baseUrl: https://<my_developer_hub_url>
  cors:
    origin: https://<my_developer_hub_url>
```

- Optionally, enter your configuration such as:
  - Authentication in Red Hat Developer Hub .
  - Authorization in Red Hat Developer Hub .
  - Customization.
  - Configure your OpenShift Container Platform integration .

3. Provision your custom configuration files to your OpenShift Container Platform cluster.

   a. Create the *<my-rhdh-project>* project aimed at containing your Developer Hub instance.

   ```
   $ oc create namespace my-rhdh-project
   ```

   Alternatively, create the project by using the web console .

   b. Provision your **app-config.yaml** file to the **my-rhdh-app-config** config map in the *<my-rhdh-project>* project.

   ```
   $ oc create configmap my-rhdh-app-config --from-file=app-config.yaml --
   namespace=my-rhdh-project
   ```

   Alternatively, create the config map by using the web console .

   c. Provision your ***<my_product_secrets>*.txt** file to the ***<my_product_secrets>*** secret in the *<my-rhdh-project>* project.

   ```
   $ oc create secret generic <my_product_secrets> --from-
   file=<my_product_secrets>.txt --namespace=my-rhdh-project
   ```

   Alternatively, create the secret by using the web console .

NOTE

<my_product_secrets> is your preferred Developer Hub secret name, specifying the identifier for your secret configuration within Developer Hub.

Next steps

Consider provisioning additional config maps and secrets:

- To use an external PostgreSQL database, provision your PostgreSQL database secrets.
- To enable dynamic plugins, provision your dynamic plugins config map .
- To configure authorization by using external files, provision your RBAC policies config map .

1.2. USING THE RED HAT DEVELOPER HUB OPERATOR TO RUN DEVELOPER HUB WITH YOUR CUSTOM CONFIGURATION

To use the Developer Hub Operator to run Red Hat Developer Hub with your custom configuration, create your Backstage custom resource (CR) that:

- Mounts files provisioned in your custom config maps.
- Injects environment variables provisioned in your custom secrets.

Prerequisites

- By using the OpenShift CLI (**oc**), you have access, with developer permissions, to the OpenShift Container Platform cluster aimed at containing your Developer Hub instance.
- Your OpenShift Container Platform administrator has installed the Red Hat Developer Hub Operator in OpenShift Container Platform.
- You have provisioned your custom config maps and secrets in your ***<my-rhdh-project>*** project.

Procedure

1. Author your Backstage CR in a **my-rhdh-custom-resource.yaml** file to use your custom config maps and secrets.

Example 1.2. Minimal my-rhdh-custom-resource.yaml custom resource example

```
apiVersion: rhdh.redhat.com/v1alpha3
kind: Backstage
metadata:
  name: my-rhdh-custom-resource
spec:
  application:
    appConfig:
      mountPath: /opt/app-root/src
      configMaps:
        - name: my-rhdh-app-config
    extraEnvs:
      secrets:
        - name: <my_product_secrets>
    extraFiles:
```

```
      mountPath: /opt/app-root/src
    route:
      enabled: true
  database:
    enableLocalDb: true
```

Example 1.3. my-rhdh-custom-resource.yaml custom resource example with dynamic plugins and RBAC policies config maps, and external PostgreSQL database secrets.

```
apiVersion: rhdh.redhat.com/v1alpha3
kind: Backstage
metadata:
  name: <my-rhdh-custom-resource>
spec:
  application:
    appConfig:
      mountPath: /opt/app-root/src
      configMaps:
        - name: my-rhdh-app-config
        - name: rbac-policies
    dynamicPluginsConfigMapName: dynamic-plugins-rhdh
    extraEnvs:
      secrets:
        - name: <my_product_secrets>
        - name: my-rhdh-database-secrets
    extraFiles:
      mountPath: /opt/app-root/src
      secrets:
        - name: my-rhdh-database-certificates-secrets
          key: postgres-crt.pem, postgres-ca.pem, postgres-key.key
    route:
      enabled: true
  database:
    enableLocalDb: false
```

Mandatory fields

No fields are mandatory. You can create an empty Backstage CR and run Developer Hub with the default configuration.

Optional fields

spec.application.appConfig.configMaps

Enter your config map name list.

Example 1.4. Mount files in the my-rhdh-app-config config map.

```
spec:
  application:
    appConfig:
      mountPath: /opt/app-root/src
      configMaps:
        - name: my-rhdh-app-config
```

Example 1.5. Mount files in the my-rhdh-app-config and rbac-policies config maps.

```
spec:
  application:
    appConfig:
      mountPath: /opt/app-root/src
      configMaps:
        - name: my-rhdh-app-config
        - name: rbac-policies
```

spec.application.extraEnvs.envs

Optionally, enter your additional environment variables that are not secrets, such as your proxy environment variables.

Example 1.6. Inject your HTTP_PROXY, HTTPS_PROXY and NO_PROXY environment variables.

```
spec:
  application:
    extraEnvs:
      envs:
        - name: HTTP_PROXY
          value: 'http://10.10.10.105:3128'
        - name: HTTPS_PROXY
          value: 'http://10.10.10.106:3128'
        - name: NO_PROXY
          value: 'localhost,example.org'
```

spec.application.extraEnvs.secrets

Enter your environment variables secret name list.

Example 1.7. Inject the environment variables in your Red Hat Developer Hub secret

```
spec:
  application:
    extraEnvs:
      secrets:
        - name: <my_product_secrets>
```

Example 1.8. Inject the environment variables in the Red Hat Developer Hub and my-rhdh-database-secrets secrets

```
spec:
  application:
    extraEnvs:
```

```
secrets:
  - name: <my_product_secrets>
  - name: my-rhdh-database-secrets
```

NOTE

<my_product_secrets> is your preferred Developer Hub secret name, specifying the identifier for your secret configuration within Developer Hub.

spec.application.extraFiles.secrets

Enter your certificates files secret name and files list.

Mount the postgres-crt.pem, postgres-ca.pem, and postgres-key.key files contained in the my-rhdh-database-certificates-secrets secret

```
spec:
  application:
    extraFiles:
      mountPath: /opt/app-root/src
      secrets:
        - name: my-rhdh-database-certificates-secrets
          key: postgres-crt.pem, postgres-ca.pem, postgres-key.key
```

spec.database.enableLocalDb

Enable or disable the local PostgreSQL database.

Disable the local PostgreSQL database generation to use an external postgreSQL database

```
spec:
  database:
    enableLocalDb: false
```

On a development environment, use the local PostgreSQL database

```
spec:
  database:
    enableLocalDb: true
```

spec.deployment

Optionally, enter your deployment configuration.

1. Apply your Backstage CR to start or update your Developer Hub instance.

```
$ oc apply --filename=my-rhdh-custom-resource.yaml --namespace=my-rhdh-project
```

1.2.1. Mounting additional files in your custom configuration using the Red Hat Developer Hub Operator

You can use the Developer Hub Operator to mount extra files, such as a ConfigMap or Secret, to the container in a preferred location.

The **mountPath** field specifies the location where a ConfigMap or Secret is mounted. The behavior of the mount, whether it includes or excludes a **subPath**, depends on the specification of the **key** or **mountPath** fields.

- If **key** and **mountPath** are not specified: Each key or value is mounted as a **filename** or content with a **subPath**.
- If **key** is specified with or without **mountPath**: The specified key or value is mounted with a **subPath**.
- If only **mountPath** is specified: A directory containing all the keys or values is mounted without a **subPath**.

NOTE

- OpenShift Container Platform does not automatically update a volume mounted with **subPath**. By default, the RHDH Operator monitors these ConfigMaps or Secrets and refreshes the RHDH Pod when changes occur.
- For security purposes, Red Hat Developer Hub does not give the Operator Service Account read access to Secrets. As a result, mounting files from Secrets without specifying both mountPath and key is not supported.

Prerequisites

- You have developer permissions to access the OpenShift Container Platform cluster containing your Developer Hub instance using the OpenShift CLI (**oc**).
- Your OpenShift Container Platform administrator has installed the Red Hat Developer Hub Operator in OpenShift Container Platform.

Procedure

1. In OpenShift Container Platform, create your ConfigMap or Secret with the following YAML codes:

   Example 1.9. Minimal my-project-configmap ConfigMap example

   ```
   apiVersion: v1
   kind: ConfigMap
   metadata:
     name: my-project-configmap
   data:
     file11.txt: |
       My file11 content
     file 12.txt: |
       My file12 content
   ```

Example 1.10. Minimal Red Hat Developer Hub Secret example

```
apiVersion: v1
kind: Secret
metadata:
  name: <my_product_secrets>
StringData:
  secret11.txt: |
    secret-content
```

For more information, see Provisioning and using your custom Red Hat Developer Hub configuration.

2. Set the value of the **configMaps name** to the name of the ConfigMap or **secrets name** to the name of the Secret in your **Backstage** CR. For example:

```
spec:
  application:
    extraFiles:
      mountPath: /my/path
      configMaps:
        - name: my-project-configmap
          key: file12.txt
          mountPath: /my/my-rhdh-config-map/path
      secrets:
        - name: <my_product_secrets>
          key: secret11.txt
          mountPath: /my/my-rhdh-secret/path
```

NOTE

<my_product_secrets> is your preferred Developer Hub secret name, specifying the identifier for your secret configuration within Developer Hub.

1.3. USING THE RED HAT DEVELOPER HUB HELM CHART TO RUN DEVELOPER HUB WITH YOUR CUSTOM CONFIGURATION

You can use the Red Hat Developer Hub Helm chart to add a custom application configuration file to your OpenShift Container Platform instance.

Prerequisites

- By using the OpenShift Container Platform web console, you have access with developer permissions, to an OpenShift Container Platform project named *<my-rhdh-project>*, aimed at containing your Developer Hub instance.
- You have uploaded your custom configuration files and secrets in your ***<my-rhdh-project>*** project.

Procedure

1. Configure Helm to use your custom configuration files in Developer Hub.

a. Go to the **Helm** tab to see the list of Helm releases.

b. Click the overflow menu on the Helm release that you want to use and select **Upgrade**.

c. Use the **YAML view** to edit the Helm configuration.

d. Set the value of the **upstream.backstage.extraAppConfig.configMapRef** and **upstream.backstage.extraAppConfig.filename** parameters as follows:

Helm configuration excerpt

```
upstream:
  backstage:
    extraAppConfig:
      - configMapRef: my-rhdh-app-config
        filename: app-config.yaml
```

e. Click **Upgrade**.

Next steps

- Install Developer Hub by using Helm.

CHAPTER 2. CONFIGURING EXTERNAL POSTGRESQL DATABASES

As an administrator, you can configure and use external PostgreSQL databases in Red Hat Developer Hub. You can use a PostgreSQL certificate file to configure an external PostgreSQL instance using the Operator or Helm Chart.

NOTE

Developer Hub supports the configuration of external PostgreSQL databases. You can perform maintenance activities, such as backing up your data or configuring high availability (HA) for the external PostgreSQL databases.

By default, the Red Hat Developer Hub operator or Helm Chart creates a local PostgreSQL database. However, this configuration is not suitable for the production environments. For production deployments, disable the creation of local database and configure Developer Hub to connect to an external PostgreSQL instance instead.

2.1. CONFIGURING AN EXTERNAL POSTGRESQL INSTANCE USING THE OPERATOR

You can configure an external PostgreSQL instance using the Red Hat Developer Hub Operator. By default, the Operator creates and manages a local instance of PostgreSQL in the same namespace where you have deployed the RHDH instance. However, you can change this default setting to configure an external PostgreSQL database server, for example, Amazon Web Services (AWS) Relational Database Service (RDS) or Azure database.

Prerequisites

- You are using a supported version of PostgreSQL. For more information, see the Product life cycle page.
- You have the following details:
  - **db-host**: Denotes your PostgreSQL instance Domain Name System (DNS) or IP address
  - **db-port**: Denotes your PostgreSQL instance port number, such as **5432**
  - **username**: Denotes the user name to connect to your PostgreSQL instance
  - **password**: Denotes the password to connect to your PostgreSQL instance
- You have installed the Red Hat Developer Hub Operator.
- Optional: You have a CA certificate, Transport Layer Security (TLS) private key, and TLS certificate so that you can secure your database connection by using the TLS protocol. For more information, refer to your PostgreSQL vendor documentation.

NOTE

By default, Developer Hub uses a database for each plugin and automatically creates it if none is found. You might need the **Create Database** privilege in addition to **PSQL Database** privileges for configuring an external PostgreSQL instance.

Procedure

1. Optional: Create a certificate secret to configure your PostgreSQL instance with a TLS connection:

```
cat <<EOF | oc -n my-rhdh-project create -f -
apiVersion: v1
kind: Secret
metadata:
 name: my-rhdh-database-certificates-secrets (1)
type: Opaque
stringData:
 postgres-ca.pem: |-
  -----BEGIN CERTIFICATE-----
  <ca-certificate-key> (2)
 postgres-key.key: |-
  -----BEGIN CERTIFICATE-----
  <tls-private-key> (3)
 postgres-crt.pem: |-
  -----BEGIN CERTIFICATE-----
  <tls-certificate-key> (4)
  # ...
EOF
```

(1) Provide the name of the certificate secret.

(2) Provide the CA certificate key.

(3) Optional: Provide the TLS private key.

(4) Optional: Provide the TLS certificate key.

2. Create a credential secret to connect with the PostgreSQL instance:

```
cat <<EOF | oc -n my-rhdh-project create -f -
apiVersion: v1
kind: Secret
metadata:
 name: my-rhdh-database-secrets (1)
type: Opaque
stringData: (2)
 POSTGRES_PASSWORD: <password>
 POSTGRES_PORT: "<db-port>"
 POSTGRES_USER: <username>
 POSTGRES_HOST: <db-host>
 PGSSLMODE: <ssl-mode> # for TLS connection (3)
 NODE_EXTRA_CA_CERTS: <abs-path-to-pem-file> # for TLS connection, e.g. /opt/app-root/src/postgres-crt.pem (4)
EOF
```

(1) Provide the name of the credential secret.

(2) Provide credential data to connect with your PostgreSQL instance.

3. Optional: Provide the value based on the required Secure Sockets Layer (SSL) mode.
4. Optional: Provide the value only if you need a TLS connection for your PostgreSQL instance.

3. Create your **Backstage** custom resource (CR):

```
cat <<EOF | oc -n my-rhdh-project create -f -
apiVersion: rhdh.redhat.com/v1alpha3
kind: Backstage
metadata:
  name: <backstage-instance-name>
spec:
  database:
    enableLocalDb: false (1)
  application:
    extraFiles:
      mountPath: <path> # e g /opt/app-root/src
      secrets:
        - name: my-rhdh-database-certificates-secrets (2)
          key: postgres-crt.pem, postgres-ca.pem, postgres-key.key # key name as in my-rhdh-database-certificates-secrets Secret
    extraEnvs:
      secrets:
        - name: my-rhdh-database-secrets (3)
        # ...
```

1. Set the value of the **enableLocalDb** parameter to **false** to disable creating local PostgreSQL instances.
2. Provide the name of the certificate secret if you have configured a TLS connection.
3. Provide the name of the credential secret that you created.

> **NOTE**
>
> The environment variables listed in the **Backstage** CR work with the Operator default configuration. If you have changed the Operator default configuration, you must reconfigure the **Backstage** CR accordingly.

4. Apply the **Backstage** CR to the namespace where you have deployed the Developer Hub instance.

2.2. CONFIGURING AN EXTERNAL POSTGRESQL INSTANCE USING THE HELM CHART

You can configure an external PostgreSQL instance by using the Helm Chart. By default, the Helm Chart creates and manages a local instance of PostgreSQL in the same namespace where you have deployed the RHDH instance. However, you can change this default setting to configure an external PostgreSQL database server, for example, Amazon Web Services (AWS) Relational Database Service (RDS) or Azure database.

Prerequisites

- You are using a supported version of PostgreSQL. For more information, see the Product life cycle page.
- You have the following details:
  - **db-host**: Denotes your PostgreSQL instance Domain Name System (DNS) or IP address
  - **db-port**: Denotes your PostgreSQL instance port number, such as **5432**
  - **username**: Denotes the user name to connect to your PostgreSQL instance
  - **password**: Denotes the password to connect to your PostgreSQL instance
- You have installed the RHDH application by using the Helm Chart.
- Optional: You have a CA certificate, Transport Layer Security (TLS) private key, and TLS certificate so that you can secure your database connection by using the TLS protocol. For more information, refer to your PostgreSQL vendor documentation.

NOTE

By default, Developer Hub uses a database for each plugin and automatically creates it if none is found. You might need the **Create Database** privilege in addition to **PSQL Database** privileges for configuring an external PostgreSQL instance.

Procedure

1. Optional: Create a certificate secret to configure your PostgreSQL instance with a TLS connection:

```
cat <<EOF | oc -n <your-namespace> create -f -
apiVersion: v1
kind: Secret
metadata:
 name: my-rhdh-database-certificates-secrets (1)
type: Opaque
stringData:
 postgres-ca.pem: |-
  -----BEGIN CERTIFICATE-----
  <ca-certificate-key> (2)
 postgres-key.key: |-
  -----BEGIN CERTIFICATE-----
  <tls-private-key> (3)
 postgres-crt.pem: |-
  -----BEGIN CERTIFICATE-----
  <tls-certificate-key> (4)
  # ...
EOF
```

(1) Provide the name of the certificate secret.

(2) Provide the CA certificate key.

3. Optional: Provide the TLS private key.
4. Optional: Provide the TLS certificate key.

2. Create a credential secret to connect with the PostgreSQL instance:

```
cat <<EOF | oc -n <your-namespace> create -f -
apiVersion: v1
kind: Secret
metadata:
 name: my-rhdh-database-secrets (1)
type: Opaque
stringData: (2)
 POSTGRES_PASSWORD: <password>
 POSTGRES_PORT: "<db-port>"
 POSTGRES_USER: <username>
 POSTGRES_HOST: <db-host>
 PGSSLMODE: <ssl-mode> # for TLS connection (3)
 NODE_EXTRA_CA_CERTS: <abs-path-to-pem-file> # for TLS connection, e.g. /opt/app-root/src/postgres-crt.pem (4)
EOF
```

1. Provide the name of the credential secret.
2. Provide credential data to connect with your PostgreSQL instance.
3. Optional: Provide the value based on the required Secure Sockets Layer (SSL) mode.
4. Optional: Provide the value only if you need a TLS connection for your PostgreSQL instance.

3. Configure your PostgreSQL instance in the Helm configuration file named **values.yaml**:

```
# ...
upstream:
  postgresql:
    enabled: false  # disable PostgreSQL instance creation (1)
    auth:
      existingSecret: my-rhdh-database-secrets # inject credentials secret to Backstage (2)
  backstage:
    appConfig:
      backend:
        database:
          connection:  # configure Backstage DB connection parameters
            host: ${POSTGRES_HOST}
            port: ${POSTGRES_PORT}
            user: ${POSTGRES_USER}
            password: ${POSTGRES_PASSWORD}
            ssl:
              rejectUnauthorized: true,
              ca:
                $file: /opt/app-root/src/postgres-ca.pem
              key:
```

```
            $file: /opt/app-root/src/postgres-key.key
          cert:
            $file: /opt/app-root/src/postgres-crt.pem
extraEnvVarsSecrets:
  - my-rhdh-database-secrets # inject credentials secret to Backstage (3)
extraEnvVars:
  - name: BACKEND_SECRET
    valueFrom:
      secretKeyRef:
        key: backend-secret
        name: '{{ include "janus-idp.backend-secret-name" $ }}'
extraVolumeMounts:
  - mountPath: /opt/app-root/src/dynamic-plugins-root
    name: dynamic-plugins-root
  - mountPath: /opt/app-root/src/postgres-crt.pem
    name: postgres-crt # inject TLS certificate to Backstage cont. (4)
    subPath: postgres-crt.pem
  - mountPath: /opt/app-root/src/postgres-ca.pem
    name: postgres-ca # inject CA certificate to Backstage cont. (5)
    subPath: postgres-ca.pem
  - mountPath: /opt/app-root/src/postgres-key.key
    name: postgres-key # inject TLS private key to Backstage cont. (6)
    subPath: postgres-key.key
extraVolumes:
  - ephemeral:
      volumeClaimTemplate:
        spec:
          accessModes:
            - ReadWriteOnce
          resources:
            requests:
              storage: 1Gi
    name: dynamic-plugins-root
  - configMap:
      defaultMode: 420
      name: dynamic-plugins
      optional: true
    name: dynamic-plugins
  - name: dynamic-plugins-npmrc
    secret:
      defaultMode: 420
      optional: true
      secretName: '{{ printf "%s-dynamic-plugins-npmrc" .Release.Name }}'
  - name: postgres-crt
    secret:
      secretName: my-rhdh-database-certificates-secrets (7)
      # ...
```

1. Set the value of the **upstream.postgresql.enabled** parameter to **false** to disable creating local PostgreSQL instances.
2. Provide the name of the credential secret.
3. Provide the name of the credential secret.

4. Optional: Provide the name of the TLS certificate only for a TLS connection.
5. Optional: Provide the name of the CA certificate only for a TLS connection.
6. Optional: Provide the name of the TLS private key only if your TLS connection requires a private key.
7. Provide the name of the certificate secret if you have configured a TLS connection.

4. Apply the configuration changes in your Helm configuration file named **values.yaml**:

```
helm upgrade -n <your-namespace> <your-deploy-name> openshift-helm-charts/redhat-developer-hub -f values.yaml --version 1.6.3
```

2.3. MIGRATING LOCAL DATABASES TO AN EXTERNAL DATABASE SERVER USING THE OPERATOR

By default, Red Hat Developer Hub hosts the data for each plugin in a PostgreSQL database. When you fetch the list of databases, you might see multiple databases based on the number of plugins configured in Developer Hub. You can migrate the data from an RHDH instance hosted on a local PostgreSQL server to an external PostgreSQL service, such as AWS RDS, Azure database, or Crunchy database. To migrate the data from each RHDH instance, you can use PostgreSQL utilities, such as **pg_dump** with **psql** or **pgAdmin**.

NOTE

The following procedure uses a database copy script to do a quick migration.

Prerequisites

- You have installed the **pg_dump** and **psql** utilities on your local machine.
- For data export, you have the PGSQL user privileges to make a full dump of local databases.
- For data import, you have the PGSQL admin privileges to create an external database and populate it with database dumps.

Procedure

1. Configure port forwarding for the local PostgreSQL database pod by running the following command on a terminal:

```
oc port-forward -n <your-namespace> <pgsql-pod-name> <forward-to-port>:<forward-from-port>
```

Where:

- The **<pgsql-pod-name>** variable denotes the name of a PostgreSQL pod with the format **backstage-psql-<deployment-name>-<_index>**.
- The **<forward-to-port>** variable denotes the port of your choice to forward PostgreSQL data to.

- The **<forward-from-port>** variable denotes the local PostgreSQL instance port, such as **5432**.

  Example: Configuring port forwarding

  ```
  oc port-forward -n developer-hub backstage-psql-developer-hub-0 15432:5432
  ```

2. Make a copy of the following **db_copy.sh** script and edit the details based on your configuration:

```
#!/bin/bash

to_host=<db-service-host> (1)
to_port=5432 (2)
to_user=postgres (3)

from_host=127.0.0.1 (4)
from_port=15432 (5)
from_user=postgres (6)

allDB=("backstage_plugin_app" "backstage_plugin_auth" "backstage_plugin_catalog"
"backstage_plugin_permission" "backstage_plugin_scaffolder" "backstage_plugin_search")
(7)

for db in ${!allDB[@]};
do
  db=${allDB[$db]}
  echo Copying database: $db
  PGPASSWORD=$TO_PSW psql -h $to_host -p $to_port -U $to_user -c "create database
$db;"
  pg_dump -h $from_host -p $from_port -U $from_user -d $db | PGPASSWORD=$TO_PSW
psql -h $to_host -p $to_port -U $to_user -d $db
done
```

1. The destination host name, for example, **<db-instance-name>.rds.amazonaws.com**.
2. The destination port, such as **5432**.
3. The destination server username, for example, **postgres**.
4. The source host name, such as **127.0.0.1**.
5. The source port number, such as the **<forward-to-port>** variable.
6. The source server username, for example, **postgres**.
7. The name of databases to import in double quotes separated by spaces, for example, **("backstage_plugin_app" "backstage_plugin_auth" "backstage_plugin_catalog" "backstage_plugin_permission" "backstage_plugin_scaffolder" "backstage_plugin_search")**.

3. Create a destination database for copying the data:

```
/bin/bash TO_PSW=<destination-db-password> /path/to/db_copy.sh (1)
```

(1) The **<destination-db-password>** variable denotes the password to connect to the destination database.

NOTE

You can stop port forwarding when the copying of the data is complete. For more information about handling large databases and using the compression tools, see the Handling Large Databases section on the PostgreSQL website.

4. Reconfigure your **Backstage** custom resource (CR). For more information, see Configuring an external PostgreSQL instance using the Operator.

5. Check that the following code is present at the end of your **Backstage** CR after reconfiguration:

```
# ...
spec:
  database:
    enableLocalDb: false
  application:
  # ...
    extraFiles:
      secrets:
        - name: my-rhdh-database-certificates-secrets
          key: postgres-crt.pem # key name as in my-rhdh-database-certificates-secrets Secret
    extraEnvs:
      secrets:
        - name: my-rhdh-database-secrets
# ...
```

NOTE

Reconfiguring the **Backstage** CR deletes the corresponding **StatefulSet** and **Pod** objects, but does not delete the **PersistenceVolumeClaim** object. Use the following command to delete the local **PersistenceVolumeClaim** object:

```
oc -n developer-hub delete pvc <local-psql-pvc-name>
```

where, the **<local-psql-pvc-name>** variable is in the **data-<psql-pod-name>** format.

6. Apply the configuration changes.

Verification

1. Verify that your RHDH instance is running with the migrated data and does not contain the local PostgreSQL database by running the following command:

```
oc get pods -n <your-namespace>
```

2. Check the output for the following details:
   - The **backstage-developer-hub-xxx** pod is in running state.
   - The **backstage-psql-developer-hub-0** pod is not available.
     You can also verify these details using the **Topology** view in the OpenShift Container Platform web console.

CHAPTER 3. CONFIGURING RED HAT DEVELOPER HUB DEPLOYMENT WHEN USING THE OPERATOR

The Red Hat Developer Hub Operator exposes a **rhdh.redhat.com/v1alpha3** API Version of its custom resource (CR). This CR exposes a generic **spec.deployment.patch** field, which gives you full control over the Developer Hub Deployment resource. This field can be a fragment of the standard **apps.Deployment** Kubernetes object.

Procedure

1. Create a **Backstage** CR with the following fields:

Example

```
apiVersion: rhdh.redhat.com/v1alpha3
kind: Backstage
metadata:
  name: developer-hub
spec:
  deployment:
    patch:
      spec:
        template:
```

labels

Add labels to the Developer Hub pod.

Example adding the label my=true

```
apiVersion: rhdh.redhat.com/v1alpha3
kind: Backstage
metadata:
  name: developer-hub
spec:
  deployment:
    patch:
      spec:
        template:
          metadata:
            labels:
              my: true
```

volumes

Add an additional volume named **my-volume** and mount it under **/my/path** in the Developer Hub application container.

Example additional volume

```
apiVersion: rhdh.redhat.com/v1alpha3
kind: Backstage
metadata:
```

```
  name: developer-hub
spec:
  deployment:
    patch:
      spec:
        template:
          spec:
            containers:
              - name: backstage-backend
                volumeMounts:
                  - mountPath: /my/path
                    name: my-volume
            volumes:
              - ephemeral:
                  volumeClaimTemplate:
                    spec:
                      storageClassName: "special"
                name: my-volume
```

Replace the default **dynamic-plugins-root** volume with a persistent volume claim (PVC) named **dynamic-plugins-root**. Note the **$patch: replace** directive, otherwise a new volume will be added.

Example **dynamic-plugins-root** volume replacement

```
apiVersion: rhdh.redhat.com/v1alpha3
kind: Backstage
metadata:
  name: developer-hub
spec:
  deployment:
    patch:
      spec:
        template:
          spec:
            volumes:
              - $patch: replace
                name: dynamic-plugins-root
                persistentVolumeClaim:
                  claimName: dynamic-plugins-root
```

cpu request

Set the CPU request for the Developer Hub application container to 250m.

Example CPU request

```
apiVersion: rhdh.redhat.com/v1alpha3
kind: Backstage
metadata:
  name: developer-hub
spec:
  deployment:
    patch:
      spec:
        template:
          spec:
```

```
        containers:
          - name: backstage-backend
            resources:
              requests:
                cpu: 250m
```

my-sidecar container

Add a new **my-sidecar** sidecar container into the Developer Hub Pod.

Example side car container

```
apiVersion: rhdh.redhat.com/v1alpha3
kind: Backstage
metadata:
  name: developer-hub
spec:
  deployment:
    patch:
      spec:
        template:
          spec:
            containers:
              - name: my-sidecar
                image: quay.io/my-org/my-sidecar:latest
```

Additional resources

- To learn more about merging, see Strategic Merge Patch.

CHAPTER 4. CONFIGURING READONLYROOTFILESYSTEM IN RED HAT DEVELOPER HUB

The Red Hat Developer Hub deployment consists of two containers: an **initContainer** that installs the Dynamic Plugins, and a backend container that runs the application. The **initContainer** has the **readOnlyRootFilesystem** option enabled by default. To enable this option on the backend container, you must either have permission to deploy resources through Helm or to create or update a CR for Operator-backed deployments. You can manually configure the **readOnlyRootFilesystem** option on the backend container by using the following methods:

- The Red Hat Developer Hub Operator
- The Red Hat Developer Hub Helm chart

4.1. CONFIGURING THE READONLYROOTFILESYSTEM OPTION IN A RED HAT DEVELOPER HUB OPERATOR DEPLOYMENT

When you are deploying Developer Hub using the Operator, you must specify a **patch** for the **deployment** in your **Backstage** custom resource (CR) that applies the **readOnlyRootFilesystem** option to the **securityContext** section in the Developer Hub backend container.

Procedure

1. In your **Backstage** CR, add the **securityContext** specification. For example:

```
spec:
  deployment:
    patch:
      spec:
        template:
          spec:
            containers:
              - name: backstage-backend (1)
                securityContext:
                  readOnlyRootFilesystem: true
```

(1) Name of the main container defined in the Operator default configuration.

4.2. CONFIGURING THE READONLYROOTFILESYSTEM OPTION IN A RED HAT DEVELOPER HUB HELM CHART DEPLOYMENT

Procedure

1. In your **values.yaml** file, add the **readOnlyRootFilesystem: true** line to the **containerSecurityContext** section. For example:

```
upstream:
  backstage:
    containerSecurityContext:
      readOnlyRootFilesystem: true
```

CHAPTER 5. CONFIGURING HIGH AVAILABILITY IN RED HAT DEVELOPER HUB

High availability (HA) is a system design approach that ensures a service remains continuously accessible, even during failures of individual components, by eliminating single points of failure. It introduces redundancy and failover mechanisms to minimize downtime and maintain operational continuity.

Red Hat Developer Hub supports HA deployments on Red Hat OpenShift Container Platform and Azure Kubernetes Service. The HA deployments enable more resilient and reliable service availability across supported environments.

In a single instance deployment, if a failure occurs, whether due to software crashes, hardware issues, or other unexpected disruptions, it would make the entire service unavailable, interrupting development workflows and access to key resources.

With HA enabled, you can scale the number of backend replicas to introduce redundancy. This setup ensures that if one pod or component fails, others continue to serve requests without disruption. The built-in load balancer manages ingress traffic and distributes the load across the available pods. Meanwhile, the RHDH backend manages concurrent requests and resolves resource-level conflicts effectively.

As an administrator, you can configure high availability by adjusting replica values in your configuration file:

- If you installed using the Operator, configure the replica values in your **Backstage** custom resource.
- If you used the Helm chart, set the replica values in the Helm configuration.

5.1. CONFIGURING HIGH AVAILABILITY IN A RED HAT DEVELOPER HUB OPERATOR DEPLOYMENT

RHDH instances that are deployed with the Operator use configurations in the **Backstage** custom resource. In the **Backstage** custom resource, the default value for the **replicas** field is **1**. If you want to configure your RHDH instance for high availability, you must set **replicas** to a value greater than **1**.

Procedure

- In your **Backstage** custom resource, set **replicas** to a value greater than **1**. For example:

```
apiVersion: rhdh.redhat.com/v1alpha3
kind: Backstage
metadata:
  name: <your_yaml_file>
spec:
  deployment:
    patch:
      spec:
        ...
        replicas: <replicas_value> (1)
        ...
```

1. Set the number of replicas based on the number of backup instances that you want to configure.

5.2. CONFIGURING HIGH AVAILABILITY IN A RED HAT DEVELOPER HUB HELM CHART DEPLOYMENT

When you are deploying Developer Hub using the Helm chart, you must set **replicas** to a value greater than **1** in your Helm chart. The default value for **replicas** is **1**.

Procedure

To configure your Developer Hub Helm chart for high availability, complete the following step:

- In your Helm chart configuration file, set **replicas** to a value greater than **1**. For example:

```
upstream:
  backstage:
    replicas: <replicas_value> (1)
```

1. Set the number of replicas based on the number of backup instances that you want to configure.

CHAPTER 6. RUNNING RED HAT DEVELOPER HUB BEHIND A CORPORATE PROXY

In a network restricted environment, configure Red Hat Developer Hub to use your proxy to access remote network resources.

You can run the Developer Hub application behind a corporate proxy by setting any of the following environment variables before starting the application:

HTTP_PROXY

Denotes the proxy to use for HTTP requests.

HTTPS_PROXY

Denotes the proxy to use for HTTPS requests.

NO_PROXY

Set the environment variable to bypass the proxy for certain domains. The variable value is a comma-separated list of hostnames or IP addresses that can be accessed without the proxy, even if one is specified.

6.1. UNDERSTANDING THE NO_PROXY EXCLUSION RULES

NO_PROXY is a comma or space-separated list of hostnames or IP addresses, with optional port numbers. If the input URL matches any of the entries listed in **NO_PROXY**, a direct request fetches that URL, for example, bypassing the proxy settings.

NOTE

The default value for **NO_PROXY** in RHDH is **localhost,127.0.0.1**. If you want to override it, include at least **localhost** or **localhost:7007** in the list. Otherwise, the RHDH backend might fail.

Matching follows the rules below:

- **NO_PROXY=*** will bypass the proxy for all requests.
- Space and commas might separate the entries in the **NO_PROXY** list. For example, **NO_PROXY="localhost,example.com"**, or **NO_PROXY="localhost example.com"**, or **NO_PROXY="localhost, example.com"** would have the same effect.
- If **NO_PROXY** contains no entries, configuring the **HTTP(S)_PROXY** settings makes the backend send all requests through the proxy.
- The backend does not perform a DNS lookup to determine if a request should bypass the proxy or not. For example, if DNS resolves **example.com** to **1.2.3.4**, setting **NO_PROXY=1.2.3.4** has no effect on requests sent to **example.com**. Only requests sent to the IP address **1.2.3.4** bypass the proxy.
- If you add a port after the hostname or IP address, the request must match both the host/IP and port to bypass the proxy. For example, **NO_PROXY=example.com:1234** would bypass the proxy for requests to **http(s)://example.com:1234**, but not for requests on other ports, like **http(s)://example.com**.

- If you do not specify a port after the hostname or IP address, all requests to that host/IP address will bypass the proxy regardless of the port. For example, **NO_PROXY=localhost** would bypass the proxy for requests sent to URLs like **http(s)://localhost:7077** and **http(s)://localhost:8888**.
- IP Address blocks in CIDR notation will not work. So setting **NO_PROXY=10.11.0.0/16** will not have any effect, even if the backend sends a request to an IP address in that block.
- Supports only IPv4 addresses. IPv6 addresses like **::1** will not work.
- Generally, the proxy is only bypassed if the hostname is an exact match for an entry in the **NO_PROXY** list. The only exceptions are entries that start with a dot (**.**) or with a wildcard (*****). In such a case, bypass the proxy if the hostname ends with the entry.

NOTE

List the domain and the wildcard domain if you want to exclude a given domain and all its subdomains. For example, you would set **NO_PROXY=example.com,.example.com** to bypass the proxy for requests sent to **http(s)://example.com** and **http(s)://subdomain.example.com**.

6.2. CONFIGURING PROXY INFORMATION IN OPERATOR DEPLOYMENT

For Operator-based deployment, the approach you use for proxy configuration is based on your role:

- As a cluster administrator with access to the Operator namespace, you can configure the proxy variables in the Operator's default ConfigMap file. This configuration applies the proxy settings to all the users of the Operator.
- As a developer, you can configure the proxy variables in a custom resource (CR) file. This configuration applies the proxy settings to the RHDH application created from that CR.

Prerequisites

- You have installed the Red Hat Developer Hub application.

Procedure

1. Perform one of the following steps based on your role:
   - As an administrator, set the proxy information in the Operator's default ConfigMap file:
     a. Search for a ConfigMap file named **backstage-default-config** in the default namespace **rhdh-operator** and open it.
     b. Find the **deployment.yaml** key.
     c. Set the value of the **HTTP_PROXY**, **HTTPS_PROXY**, and **NO_PROXY** environment variables in the **Deployment** spec as shown in the following example:

     Example: Setting proxy variables in a ConfigMap file

     ```
     # Other fields omitted
     ```

```
deployment.yaml: |-
  apiVersion: apps/v1
  kind: Deployment
  spec:
    template:
      spec:
        # Other fields omitted
        initContainers:
          - name: install-dynamic-plugins
            # command omitted
            env:
              - name: NPM_CONFIG_USERCONFIG
                value: /opt/app-root/src/.npmrc.dynamic-plugins
              - name: HTTP_PROXY
                value: 'http://10.10.10.105:3128'
              - name: HTTPS_PROXY
                value: 'http://10.10.10.106:3128'
              - name: NO_PROXY
                value: 'localhost,example.org'
            # Other fields omitted
        containers:
          - name: backstage-backend
            # Other fields omitted
            env:
              - name: APP_CONFIG_backend_listen_port
                value: "7007"
              - name: HTTP_PROXY
                value: 'http://10.10.10.105:3128'
              - name: HTTPS_PROXY
                value: 'http://10.10.10.106:3128'
              - name: NO_PROXY
                value: 'localhost,example.org'
```

- As a developer, set the proxy information in your **Backstage** CR file as shown in the following example:

  Example: Setting proxy variables in a CR file

```
spec:
  # Other fields omitted
  application:
    extraEnvs:
      envs:
        - name: HTTP_PROXY
          value: 'http://10.10.10.105:3128'
        - name: HTTPS_PROXY
          value: 'http://10.10.10.106:3128'
        - name: NO_PROXY
          value: 'localhost,example.org'
```

2. Save the configuration changes.

6.3. CONFIGURING PROXY INFORMATION IN HELM DEPLOYMENT

For Helm-based deployment, either a developer or a cluster administrator with permissions to create resources in the cluster can configure the proxy variables in a **values.yaml** Helm configuration file.

Prerequisites

- You have installed the Red Hat Developer Hub application.

Procedure

1. Set the proxy information in your Helm configuration file:

```
upstream:
  backstage:
    extraEnvVars:
      - name: HTTP_PROXY
        value: '<http_proxy_url>'
      - name: HTTPS_PROXY
        value: '<https_proxy_url>'
      - name: NO_PROXY
        value: '<no_proxy_settings>'
```

Where,

<http_proxy_url>

Denotes a variable that you must replace with the HTTP proxy URL.

<https_proxy_url>

Denotes a variable that you must replace with the HTTPS proxy URL.

<no_proxy_settings>

Denotes a variable that you must replace with comma-separated URLs, which you want to exclude from proxying, for example, **foo.com,baz.com**.

Example: Setting proxy variables using Helm Chart

```
upstream:
  backstage:
    extraEnvVars:
      - name: HTTP_PROXY
        value: 'http://10.10.10.105:3128'
      - name: HTTPS_PROXY
        value: 'http://10.10.10.106:3128'
      - name: NO_PROXY
        value: 'localhost,example.org'
```

2. Save the configuration changes.

CHAPTER 7. CONFIGURING AN RHDH INSTANCE WITH A TLS CONNECTION IN KUBERNETES

You can configure a RHDH instance with a Transport Layer Security (TLS) connection in a Kubernetes cluster, such as an Azure Red Hat OpenShift (ARO) cluster, any cluster from a supported cloud provider, or your own cluster with proper configuration. Transport Layer Security (TLS) ensures a secure connection for the RHDH instance with other entities, such as third-party applications, or external databases. However, you must use a public Certificate Authority (CA)-signed certificate to configure your Kubernetes cluster.

Prerequisites

- You have set up an Azure Red Hat OpenShift (ARO) cluster with a public CA-signed certificate. For more information about obtaining CA certificates, refer to your vendor documentation.
- You have created a namespace and setup a service account with proper read permissions on resources.

  Example: Kubernetes manifest for role-based access control

  ```
  apiVersion: rbac.authorization.k8s.io/v1
  kind: ClusterRole
  metadata:
    name: backstage-read-only
  rules:
    - apiGroups:
        - '*'
      resources:
        - pods
        - configmaps
        - services
        - deployments
        - replicasets
        - horizontalpodautoscalers
        - ingresses
        - statefulsets
        - limitranges
        - resourcequotas
        - daemonsets
      verbs:
        - get
        - list
        - watch
  #...
  ```

- You have obtained the secret and the service CA certificate associated with your service account.
- You have created some resources and added annotations to them so they can be discovered by the Kubernetes plugin. You can apply these Kubernetes annotations:
  - **backstage.io/kubernetes-id** to label components
  - **backstage.io/kubernetes-namespace** to label namespaces

Procedure

1. Enable the Kubernetes plugins in the **dynamic-plugins-rhdh.yaml** file:

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: dynamic-plugins-rhdh
data:
  dynamic-plugins.yaml: |
    includes:
      - dynamic-plugins.default.yaml
    plugins:
      - package: ./dynamic-plugins/dist/backstage-plugin-kubernetes-backend-dynamic
        disabled: false (1)
      - package: ./dynamic-plugins/dist/backstage-plugin-kubernetes
        disabled: false (2)
        # ...
```

(1) Set the value to **false** to enable the **backstage-plugin-kubernetes-backend-dynamic** plugin.

(2) Set the value to **false** to enable the **backstage-plugin-kubernetes** plugin.

> **NOTE**
>
> The **backstage-plugin-kubernetes** plugin is currently in Technology Preview. As an alternative, you can use the **./dynamic-plugins/dist/backstage-plugin-topology-dynamic** plugin, which is Generally Available (GA).

2. Set the Kubernetes cluster details and configure the catalog sync options in the **app-config.yaml** configuration file.

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: my-rhdh-app-config
data:
  "app-config.yaml": |
  # ...
  catalog:
    rules:
      - allow: [Component, System, API, Resource, Location]
    providers:
      kubernetes:
        openshift:
          cluster: openshift
          processor:
            namespaceOverride: default
            defaultOwner: guests
          schedule:
            frequency:
              seconds: 30
            timeout:
```

```
          seconds: 5
kubernetes:
  serviceLocatorMethod:
    type: 'multiTenant'
  clusterLocatorMethods:
    - type: 'config'
      clusters:
        - url: <target-cluster-api-server-url> (1)
          name: openshift
          authProvider: 'serviceAccount'
          skipTLSVerify: false (2)
          skipMetricsLookup: true
          dashboardUrl: <target-cluster-console-url> (3)
          dashboardApp: openshift
          serviceAccountToken: ${K8S_SERVICE_ACCOUNT_TOKEN} (4)
          caData: ${K8S_CONFIG_CA_DATA} (5)
          # ...
```

1. The base URL to the Kubernetes control plane. You can run the **kubectl cluster-info** command to get the base URL.
2. Set the value of this parameter to **false** to enable the verification of the TLS certificate.
3. Optional: The link to the Kubernetes dashboard managing the ARO cluster.
4. Optional: Pass the service account token using a **K8S_SERVICE_ACCOUNT_TOKEN** environment variable that you define in your ***<my_product_secrets>*** secret.
5. Pass the CA data using a **K8S_CONFIG_CA_DATA** environment variable that you define in your ***<my_product_secrets>*** secret.

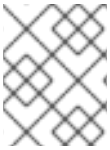
3. Save the configuration changes.

Verification

1. Run the RHDH application to import your catalog:

```
kubectl -n rhdh-operator get pods -w
```

2. Verify that the pod log shows no errors for your configuration.
3. Go to **Catalog** and check the component page in the Developer Hub instance to verify the cluster connection and the presence of your created resources.



NOTE

If you encounter connection errors, such as certificate issues or permissions, check the message box in the component page or view the logs of the pod.

CHAPTER 8. USING THE DYNAMIC PLUGINS CACHE

8.1. USING THE DYNAMIC PLUGINS CACHE

The dynamic plugins cache in Red Hat Developer Hub (RHDH) enhances the installation process and reduces platform boot time by storing previously installed plugins. If the configuration remains unchanged, this feature prevents the need to re-download plugins on subsequent boots.

When you enable dynamic plugins cache:

- The system calculates a checksum of each plugin's YAML configuration (excluding **pluginConfig**).
- The checksum is stored in a file named **dynamic-plugin-config.hash** within the plugin's directory.
- During boot, if a plugin's package reference matches the previous installation and the checksum is unchanged, the download is skipped.
- Plugins that are disabled since the previous boot are automatically removed.

NOTE

To enable the dynamic plugins cache in RHDH, the plugins directory **dynamic-plugins-root** must be a persistent volume.

8.2. CREATING A PVC FOR THE DYNAMIC PLUGIN CACHE BY USING THE OPERATOR

For operator-based installations, you must manually create the persistent volume claim (PVC) by replacing the default **dynamic-plugins-root** volume with a PVC named **dynamic-plugins-root**.

Prerequisites

- You have installed Red Hat Developer Hub on OpenShift Container Platform using the Red Hat Developer Hub Operator.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. Create the persistent volume definition and save it to a file, such as **pvc.yaml**. For example:

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: dynamic-plugins-root
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
```

NOTE

This example uses **ReadWriteOnce** as the access mode which prevents multiple replicas from sharing the PVC across different nodes. To run multiple replicas on different nodes, depending on your storage driver, you must use an access mode such as **ReadWriteMany**.

2. To apply this PVC to your cluster, run the following command:

```
oc apply -f pvc.yaml
```

3. Replace the default **dynamic-plugins-root** volume with a PVC named **dynamic-plugins-root**. For example:

```
apiVersion: rhdh.redhat.com/v1alpha3
kind: Backstage
metadata:
  name: developer-hub
spec:
  deployment:
    patch:
      spec:
        template:
          spec:
            volumes:
              - $patch: replace
                name: dynamic-plugins-root
                persistentVolumeClaim:
                  claimName: dynamic-plugins-root
```

NOTE

To avoid adding a new volume, you must use the **$patch: replace** directive.

8.3. CREATING A PVC FOR THE DYNAMIC PLUGIN CACHE USING THE HELM CHART

For Helm chart installations, if you require the dynamic plugin cache to persist across pod restarts, you must create a persistent volume claim (PVC) and configure the Helm chart to use it.

Prerequisites

- You have installed Red Hat Developer Hub using the Helm chart.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. Create the persistent volume definition. For example:

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
```

```
  name: dynamic-plugins-root
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
```

NOTE

This example uses **ReadWriteOnce** as the access mode which prevents multiple replicas from sharing the PVC across different nodes. To run multiple replicas on different nodes, depending on your storage driver, you must use an access mode such as **ReadWriteMany**.

2. To apply this PVC to your cluster, run the following command:

```
oc apply -f pvc.yaml
```

3. Configure the Helm chart to use the PVC. For example:

```
upstream:
  backstage:
    extraVolumes:
      - name: dynamic-plugins-root
        persistentVolumeClaim:
          claimName: dynamic-plugins-root
      - name: dynamic-plugins
        configMap:
          defaultMode: 420
          name: '{{ printf "%s-dynamic-plugins" .Release.Name }}'
          optional: true
      - name: dynamic-plugins-npmrc
        secret:
          defaultMode: 420
          optional: true
          secretName: '{{ printf "%s-dynamic-plugins-npmrc" .Release.Name }}'
      - name: dynamic-plugins-registry-auth
        secret:
          defaultMode: 416
          optional: true
          secretName: '{{ printf "%s-dynamic-plugins-registry-auth" .Release.Name }}'
      - name: npmcacache
        emptyDir: {}
      - name: temp
        emptyDir: {}
```

NOTE

When you configure the Helm chart to use the PVC, you must also include the **extraVolumes** defined in the default Helm chart.

8.4. CONFIGURING THE DYNAMIC PLUGINS CACHE

You can set the following optional dynamic plugin cache parameters in your **dynamic-plugins.yaml** file:

- **forceDownload**: Set the value to **true** to force a reinstall of the plugin, bypassing the cache. The default value is **false**.
- **pullPolicy**: Similar to the **forceDownload** parameter and is consistent with other image container platforms. You can use one of the following values for this key:
  - **Always**: This value compares the image digest in the remote registry and downloads the artifact if it has changed, even if the plugin was previously downloaded.
  - **IfNotPresent**: This value downloads the artifact if it is not already present in the dynamic-plugins-root folder, without checking image digests.

NOTE

The **pullPolicy** setting is also applied to the NPM downloading method, although **Always** will download the remote artifact without a digest check. The existing **forceDownload** option remains functional, however, the **pullPolicy** option takes precedence. The **forceDownload** option may be deprecated in a future Developer Hub release.

Example dynamic-plugins.yaml file configuration to download the remote artifact without a digest check:

```
plugins:
  - disabled: false
    pullPolicy: Always
    package: 'oci://quay.io/example-org/example-plugin:v1.0.0!internal-backstage-plugin-example'
```

CHAPTER 9. CONFIGURING DEFAULT MOUNTS FOR SECRETS AND PVCS

You can configure Persistent Volume Claims (PVCs) and Secrets mount in your Red Hat Developer Hub deployment. Use annotations to define the custom mount paths and specify the containers to mount them to.

9.1. CONFIGURING MOUNT PATHS FOR SECRETS AND PVCS

By default, the mount path is the working directory of the Developer Hub container. If you do not define the mount path, it defaults to **/opt/app-root/src**.

Procedure

1. To specify a PVC mount path, add the **rhdh.redhat.com/mount-path** annotation to your configuration file as shown in the following example:

   Example specifying where the PVC mounts

   ```
   apiVersion: v1
   kind: PersistentVolumeClaim
   metadata:
     name: <my_claim>
     annotations:
       rhdh.redhat.com/mount-path: /mount/path/from/annotation
   ```

   where:

   rhdh.redhat.com/mount-path

   Specifies which mount path the PVC mounts to (in this case, **/mount/path/from/annotation** directory).

   <my_claim>

   Specifies the PVC to mount.

2. To specify a Secret mount path, add the **rhdh.redhat.com/mount-path** annotation to your configuration file as shown in the following example:

   Example specifying where the Secret mounts

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     name: <my_secret>
     annotations:
       rhdh.redhat.com/mount-path: /mount/path/from/annotation
   ```

   where:

   <my_secret>

   Specifies the Secret name.

9.2. MOUNTING SECRETS AND PVCS TO SPECIFIC CONTAINERS

By default, Secrets and PVCs mount only to the Red Hat Developer Hub **backstage-backend** container. You can add the **rhdh.redhat.com/containers** annotation to your configuration file to specify the containers to mount to.

Procedure

1. To mount Secrets to **all** containers, set the **rhdh.redhat.com/containers** annotation to ***** in your configuration file:

   Example mounting to all containers

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     name: <my_secret>
     annotations:
       rhdh.redhat.com/containers: *
   ```

   IMPORTANT

   Set **rhdh.redhat.com/containers** to ***** to mount it to all containers in the deployment.

2. To mount to specific containers, separate the names with commas:

   Example separating the list of containers

   ```
   apiVersion: v1
   kind: PersistentVolumeClaim
   metadata:
     name: <my_claim>
     annotations:
       rhdh.redhat.com/containers: "init-dynamic-plugins,backstage-backend"
   ```

   NOTE

   This configuration mounts the ***<my_claim>*** PVC to the **init-dynamic-plugins** and **backstage-backend** containers.

CHAPTER 10. ENABLING THE RED HAT DEVELOPER HUB PLUGIN ASSETS CACHE

By default, Red Hat Developer Hub does not cache plugin assets. You can use a Redis cache store to improve Developer Hub performance and reliability. Configured plugins in Developer Hub receive dedicated cache connections, which are powered by the Keyv Redis client.

Prerequisites

- You have installed Red Hat Developer Hub.
- You have an active Redis server. For more information on setting up an external Redis server, see the **official Redis documentation**.

Procedure

1. Enable the Developer Hub cache by defining Redis as the cache store type and entering your Redis server connection URL in your **app-config.yaml** file.

   app-config.yaml file fragment

   ```
   backend:
     cache:
       store: redis
       connection: redis://user:pass@cache.example.com:6379
   ```

2. Enable the cache for Techdocs by adding the **techdocs.cache.ttl** setting in your **app-config.yaml** file. This setting specifies how long, in milliseconds, a statically built asset should stay in the cache.

   app-config.yaml file fragment

   ```
   techdocs:
     cache:
       ttl: 3600000
   ```

TIP

Optionally, enable the cache for unsupported plugins that support this feature. See their respective documentation for details.